

Tutorial Sheet - Topic 10

Programming Fundamentals 2

Abstraction & Interfaces in Java

Exercise 1 -- Spot the Difference (Abstract vs Concrete)

Consider the following two class declarations:

```
public abstract class Animal {  
    public abstract void makeSound();  
}  
  
public class Dog extends Animal {  
    public void makeSound() {  
        System.out.println("Bark");  
    }  
}
```

Tasks:

1. Which class is **abstract**?
 2. Can you create an object of `Animal`? Why/why not?
 3. Why is `Dog` considered a **concrete** class?
 4. What would happen if `Dog` did **not** implement `makeSound()`?
-

Exercise 2 -- Writing Abstract Classes

You are designing a system for shapes.

Tasks:

1. Create an **abstract class** called `Shape` with:
 - an abstract method `calculateArea()`
 2. Create a class `Rectangle` that extends `Shape` and implements `calculateArea()`
 3. Create a class `Circle` that extends `Shape` and implements `calculateArea()`
-

Exercise 3 -- Why Abstract?

Given this method in a superclass:

```
public abstract void display();
```

Tasks:

1. Why might a designer choose to make this method abstract?
2. What does this force subclasses to do?
3. Give a real-world example where this design makes sense.

Exercise 4 -- The "Deadly Diamond of Death" (Conceptual)

```
class A {
    void doSomething() { System.out.println("A"); }
}

class B extends A {
    void doSomething() { System.out.println("B"); }
}

class C extends A {
    void doSomething() { System.out.println("C"); }
}

// Not allowed in Java
class D extends B, C {
}
```

Tasks:

1. What problem occurs in class `D`?
2. Why can Java not decide which `doSomething()` to use?
3. Explain why this leads to the "Deadly Diamond of Death".
4. How does Java avoid this problem?

Exercise 5 -- Introduction to Interfaces

```
interface IMovable {
    void move();
}
```

Tasks:

1. Create a class `Car` that implements `IMovable`
2. Create a class `Person` that implements `IMovable`
3. Give different implementations of `move()` for each

Exercise 6 -- Interfaces as Contracts

```
interface IAnimal {  
    void eat();  
    void sleep();  
}
```

Tasks:

1. Create a class `Cat` that implements `IAnimal`
 2. What happens if you forget to implement one method?
 3. What keyword must you use if you don't implement all methods?
-

Exercise 7 -- Multiple Interfaces

```
interface IFlyable {  
    void fly();  
}  
  
interface ISwimmable {  
    void swim();  
}
```

Tasks:

1. Create a class `Duck` that implements both interfaces
 2. Implement both methods
 3. Explain why Java allows this (but not multiple class inheritance)
-

Exercise 8 -- Interfaces as Types

```
interface IMammal {  
    void eat();  
}  
  
class Dog implements IMammal {  
    public void eat() {  
        System.out.println("Dog eating");  
    }  
}
```

Tasks:

1. Write: `IMammal m = new Dog();`
 2. Why is this useful?
 3. What happens if you replace `Dog` with another implementation?
-

Exercise 9 -- Collections and Interfaces

```
List<String> names = new ArrayList<>();
```

Tasks:

1. Why use `List` instead of `ArrayList`?
 2. Replace with `LinkedList` -- what changes?
 3. Why is this more maintainable?
-

Exercise 10 -- Design Exercise

Design a **media system**.

Requirements:

- All media must be playable → `play()`
- Some media can be downloaded → `download()`

Tasks:

1. Create interfaces `Playable` and `Downloadable`
 2. Create classes:
 - `Song`
 - `LiveStream`
 3. Test in `main()`
-

Challenge Exercise

Design a **vehicle system** using: - an abstract class `Vehicle` - at least one interface

Tasks:

1. Design structure
 2. Implement two classes
 3. Justify your design choices
-